What is NoSQL?

NoSQL (originally referring to "non SQL" or "non relational") is a database that provides a mechanism for storage and retrieval of data modeled in means other than the tabular relations used in relational databases.

- Such databases came into existence in the late 1960s
- Gained popularity as "NoSQL" in the early twenty-first century
- Used in real-time web applications and big data
- Features include simplicity of design, horizontal scaling, and fine control over availability
- Different data structures make some operations faster than in relational databases

Advantages of NoSQL

- High scalability
- High availability
- Flexible schema design
- Efficient horizontal scaling
- Better performance for certain workloads

Disadvantages of NoSQL

- Narrow focus (specialized for specific use cases)
- Mostly open source (may lack enterprise support)
- Management challenges
- Limited GUI tools compared to RDBMS
- Backup can be complex
- Large document size limitations

Types of NoSQL Databases

- **Key-value store:** Memcached, Redis, Coherence
- **Tabular:** HBase, BigTable, Accumulo
- **Document-based:** MongoDB, CouchDB, Cloudant

MongoDB Introduction

MongoDB is an open-source document-oriented database designed to store large-scale data and work with that data efficiently.

- Categorized under NoSQL (Not only SQL) databases
- Developed and managed by MongoDB.Inc under SSPL (Server Side Public License)
- Initially released in February 2009
- Provides official driver support for popular languages (C, C++, C#, Java, Python, etc.)
- Used by companies like Facebook, Nokia, eBay, Adobe, Google

Features of MongoDB

- Schema-less Database
- Document Oriented
- Indexing
- Scalability
- Replication
- Aggregation
- High Performance

MongoDB Structure

```
Database → Collections → Documents → Fields (key-value pairs)
```

Documents are created using fields which are key-value pairs (similar to columns in relational databases).

BSON Format

MongoDB stores data in BSON (Binary JSON) format:

- BSON = Binary representation of JSON documents
- Server converts JSON to BSON for efficient storage and querying
- Supports nested data (complex relations within same document)
- Maximum document size: 16MB

MongoDB	RDBMS
Non-relational, document-oriented	Relational
Suitable for hierarchical data	Not suitable for hierarchical data
Dynamic schema	Predefined schema
CAP theorem (Consistency, Availability, Partition tolerance)	ACID properties (Atomicity, Consistency, Isolation,
Generally faster performance	Generally slower performance

MongoDB vs RDBMS

MongoDB Environment Setup

Install MongoDB Community Server

MongoDB Community Server includes the database engine used for local development.

- Go to the official MongoDB website
- Select your operating system (Windows, Mac, Linux)
- Download & Install

```
Website: https://www.mongodb.com/try/download/community

Steps:
1. Choose Version (Current Stable Version)
2. Select Platform (Windows/macOS/Linux)
3. Choose Installer (MSI for Windows)
```

```
4. Install with "Complete" setup
5. Make sure "Install MongoDB as a Service" is checked
```

Install MongoDB Shell (mongosh)

MongoDB Shell (mongosh) is the command-line tool used to interact with the database.

```
Download Mongosh:
https://www.mongodb.com/try/download/shell

Steps:
1. Choose your OS (Windows/macOS/Linux)
2. Download ZIP/Installer
3. Extract or Install
4. Add to PATH (Windows only)
```

Check installation:

```
mongosh --version
```

Install MongoDB Compass (GUI Tool)

Compass is a visual GUI tool to interact with your MongoDB databases.

```
Download Compass:
https://www.mongodb.com/try/download/compass

Steps:
1. Select OS (Windows/macOS/Linux)
2. Download the installer
3. Install Compass
4. Open Compass and connect using:
   mongodb://localhost:27017
```

Start MongoDB on Local Machine

MongoDB usually starts as a background service automatically.

Check service status (Windows):

```
services.msc  // Search in Start Menu
```

Start using shell:

```
mongosh
```

MongoDB Atlas (Cloud Database)

MongoDB Atlas is a fully managed cloud MongoDB service.

```
Steps to create Atlas account:
1. Go to https://www.mongodb.com/cloud/atlas/register
2. Create free account
3. Create new project
4. Build a Cluster → Choose Free Tier (M0)
5. Choose Cloud Provider (AWS/GCP/Azure)
6. Choose Region (Closest to India → Mumbai)
7. Create Cluster
```

Create Database User in Atlas

```
Steps:
```

```
1. Go to Database Access
2. Click "Add New Database User"
3. Enter username & password
4. Choose "Read and Write to any database"
5. Click Create User
```

Allow Network Access (IP Whitelist)

```
Steps:
1. Go to "Network Access"
2. Click "Add IP Address"
3. Choose:
   - Allow access from anywhere → 0.0.0.0/0
4. Save
```

Connect to MongoDB Atlas

```
// Use this connection string format
mongodb+srv://<username>:<password>@cluster0.xxxxx.mongodb.net/

// Example connection string
mongodb+srv://mugil:password123@cluster0.abcd.mongodb.net/test
```

Connect using mongosh:

```
mongosh "mongodb+srv://mugil:password123@cluster0.abcd.mongodb.net"
```

Connect using Compass:

```
1. Open Compass
2. Click "New Connection"
3. Paste the URI:
   mongodb+srv://mugil:password123@cluster0.abcd.mongodb.net
4. Connect
```

Test Your Connection

```
// Inside mongosh
show dbs;
use test;
db.sample.insertOne({ name: "MongoDB Atlas Working!" });
```

MongoDB Data Types

- **String:** UTF-8 valid strings
- **Integer:** 32-bit or 64-bit numbers
- **Boolean:** true/false values
- **Double:** Floating-point numbers
- **Min/Max keys:** For comparing against lowest/highest BSON elements
- **Arrays:** Lists or multiple values
- **Timestamp:** For recording document modifications
- **Object:** For embedded documents
- **Null:** For storing null values
- **Symbol:** Similar to string but for language symbol types
- **Date:** Stores date/time in UNIX format
- **Object ID:** Unique document identifier (12-byte)
- **Binary data:** For storing binary data
- **Code:** JavaScript code
- **Regular expression:** For regex patterns

Operators

Comparison Operators

- `$eq`: Matches values equal to specified value
- `$ne`: Matches values not equal to specified value
- `$gt`: Matches values greater than specified value
- `$gte`: Matches values greater than or equal to specified value
- `$lt`: Matches values less than specified value
- `$lte`: Matches values less than or equal to specified value
- `$in`: Matches any values specified in an array
- `$nin`: Matches none of the values specified in an array

Logical Operators

- `$and`: Joins query clauses with logical AND
- `$or`: Joins query clauses with logical OR
- `$not`: Inverts the effect of query expressions
- `$nor`: Joins query clauses with logical NOR

```
// Example using $and
db.collection.find({
  $and: [
    {age: {$gt: 30}},
    {name: "Dhoni"}
  ]
});

// Example using $or
db.collection.find({
  $or: [
    {age: {$lt: 30}},
    {name: "Virat"}
  ]
});
```

Database & Collection Operations

Create Database

MongoDB creates a database automatically when you start using it:

```
// Switch to a new database (auto-creates it)
use schoolDB;
```

Use Database

Used to switch to an existing database:

```
// Switch to an existing database
use studentDB;
```

Drop Database

Deletes the currently selected database:

```
// Drop current database
db.dropDatabase();
```

Create Collection

Used to manually create a new collection:

```
// Create a collection
db.createCollection("students");
```

Drop Collection

Used to delete a collection from a database:

```
// Drop a specific collection
db.students.drop();
```

CRUD Operations

Create Operations

Used to insert new documents into a collection:

```
// Insert a single document
db.collection.insertOne({
  "name": "Dhoni",
  "age": 43
});

// Insert multiple documents
db.collection.insertMany([
  {"name": "Dhoni", "age": 43},
  {"name": "Virat", "age": 37}
]);
```

Read Operations

Used to retrieve documents from a collection:

```
// Find all documents
db.students.find();

// Pretty print results
db.students.find().pretty();

// Find documents with condition
db.students.find({ age: { $gt: 30 } });

// Projection: show only selected fields
db.students.find({}, { name: 1, age: 1, _id: 0 });

// Sorting results
db.students.find().sort({ age: -1 });  // Descending

// Limit results
db.students.find().limit(3);
```

```
// Count matching documents
db.students.find({ age: { $gt: 25 } }).count();
```

Update Operations

Used to modify existing documents:

```
// Update a single document
db.students.updateOne(
  { name: "Virat" },
  { $set: { age: 38 } }
);

// Update multiple documents
db.students.updateMany(
  { name: "Dhoni" },
  { $set: { age: 44 } }
);

// Replace an entire document
db.students.replaceOne(
  { name: "Dhoni" },
  { name: "MS Dhoni", age: 44 }
);
```

Delete Operations

Used to remove documents from a collection:

```
// Delete single document
db.students.deleteOne({ age: 37 });

// Delete multiple documents
db.students.deleteMany({ age: { $gt: 40 } });

// Delete all documents in a collection
db.students.deleteMany({});
```

Data Modeling

Embedded Data Model

All related data is stored in a single document (denormalized):

```
{
  _id: ObjectId("..."),
  Emp_ID: "10025AE336",
  Personal_details: {
    First_Name: "Radhika",
    Last_Name: "Sharma",
    Date_Of_Birth: "1995-09-26"
  },
  Contact: {
    email: "radhika_sharma.123@gmail.com",
    phone: "9848022338"
  },
  Address: {
    city: "Hyderabad",
    Area: "Madapur",
    State: "Telangana"
```

```
  }
}
```

Normalized Data Model

Documents reference each other (similar to relational model):

```
// Employee document
{
  _id: ObjectId("ObjectId101"),
  Emp_ID: "10025AE336"
}

// Personal details document
{
  _id: ObjectId("ObjectId102"),
  empDocID: "ObjectId101",
  First_Name: "Radhika",
  Last_Name: "Sharma",
  Date_Of_Birth: "1995-09-26"
}

// Contact document
{
  _id: ObjectId("ObjectId103"),
  empDocID: "ObjectId101",
  email: "radhika_sharma.123@gmail.com",
  phone: "9848022338"
}
```

Aggregation

Aggregation in MongoDB is used to process documents and return computed results. It is mainly used for:

- Calculating totals (sum)
- Finding averages
- Grouping data
- Filtering and transforming documents

MongoDB provides three ways to perform aggregation:

- **Aggregation Pipeline** (most used)
- **Map-Reduce**
- **Single-purpose Aggregation** (count, distinct, etc.)

Aggregation Pipeline

An aggregation pipeline consists of multiple stages. Each stage transforms the documents and passes them to the next stage.

```
db.orders.aggregate([
  { $match: { status: "A" } },                      // Filter documents
  { $group: { _id: "$customer", total: { $sum: "$amount" } } }, // Group by
customer
  { $sort: { total: -1 } }                          // Sort by total amount
desc
```

```
]);
```

Aggregation Expressions

- `$sum`: Adds values together
- `$avg`: Calculates average value
- `$min`: Returns minimum value
- `$max`: Returns maximum value
- `$push`: Pushes value into an array
- `$addToSet`: Pushes unique values (no duplicates)
- `$first`: Gets first document in group
- `$last`: Gets last document in group

Pipeline Stages

- `$project`: Selects or reshapes fields
- `$match`: Filters documents
- `$group`: Groups documents for calculations
- `$sort`: Sorts documents
- `$skip`: Skips number of documents
- `$limit`: Limits number of documents

Aggregation Examples

```
// Example dataset (users)
{
  name: "Arun",
  city: "Chennai",
  age: 28
}

// 1️⃣ $group example: Average age of users by city
db.users.aggregate([
  {
    $group: {
      _id: "$city",
      averageAge: { $avg: "$age" }
    }
  }
]);

// 2️⃣ $match example: Find users above age 30
db.users.aggregate([
  { $match: { age: { $gt: 30 } } }
]);

// 3️⃣ $sort example: Sort users by age ascending
db.users.aggregate([
  { $sort: { age: 1 } }  // 1 = ascending, -1 = descending
]);

// 4️⃣ $limit example: Get the first 2 users
db.users.aggregate([
  { $limit: 2 }
]);
```

```
// 5️⃣ $project example: Show only name and city
db.users.aggregate([
  {
    $project: {
      _id: 0,
      name: 1,
      city: 1
    }
  }
]);

// 6️⃣ Combined Example: Filter → Sort → Limit
// Get top 3 oldest users above age 30
db.users.aggregate([
  { $match: { age: { $gt: 30 } } },
  { $sort: { age: -1 } },
  { $limit: 3 }
]);
```